

The model got better, and that was the problem.

A column entered a feature table. Every test passed and accuracy went up by three and a half points. What also went up was how much the model knew about who is disabled. Only the lineage graph could see both.

+0.0355

ACCURACY GAINED

+0.0394

DISABILITY RECOVERED

12×

THE MEASURED NOISE

7

HOPS TO THE ORIGIN

Nobody owns the space between the pipelines

Around forty per cent of production machine learning failures trace back to a mismatch between what a pipeline produced and what a model expected. The bug lives in the space between teams, tools and assumptions, which is exactly why nobody owns it until the postmortem.

That space has a shape. It is a graph: raw tables feeding staging models feeding marts feeding feature tables feeding training runs feeding a deployment. Every real failure of this kind is a path through that graph that nobody looked at.

The tools built to watch models cannot see it. Model monitoring watches predictions and tells you a distribution shifted. It has no idea which dbt model changed last Tuesday, because it has never been told the warehouse exists. It can raise the alarm and it can never answer the next question.

The tools built to watch data can see it and are not looking at the model. A catalog holds the lineage, the column tags and the ownership. It does not know which model is serving, what that model trained on, or that a column arrived in the feature table between two training runs.

Ariadne is the join. It reads the DataHub lineage graph, measures what a deployed model can reconstruct from its features, and writes the finding back into DataHub as an incident on the table it concerns. The measurement supplies the alarm. The graph supplies the cause.

The one sentence version. Model observability can tell you that something changed. Ariadne can tell you which column, which file, which commit, and how many hops away it entered, because it is standing on the lineage graph rather than next to it.

What actually happened

The scenario is not invented. It runs on US Census ACS PUMS data, 378,817 person records and 159,685 households for California in 2018, pulled live through [folktables](#). Two models are trained on it, registered in MLflow and promoted to Production. One predicts income, one predicts whether a working age person holds full time work.

Somebody adds one column

Benefits administration asks for a coverage flag so onboarding can skip the enrolment prompt for people who already have cover. It is a reasonable request and the change is one line in a dbt model. No health detail, just a flag.

```
warehouse/models/marts/workforce_features.sql

household_size,
census_region,
vehicles_available,
+ -- benefits admin asked for the coverage flag so onboarding can tell who
+ -- already has cover and skip the enrolment prompt. no health detail,
+ -- just the flag
+ public_coverage_flag,
```

The training script takes every column that is not the key or the label, so the column became a feature without anyone deciding it should be one. The model retrains, registers as version 3, and is promoted.

MEASURE	VERSION 2	VERSION 3	CHANGE	
model accuracy	0.6955	0.7310	+0.0355	looks good
disability recoverable from features	0.7489	0.7883	+0.0394	12× noise
race, sex, marital status, citizenship	—	—	below noise	silent

This is the whole point. The metric that would normally catch a problem is pointing the wrong way. Accuracy improved, so every dashboard, every model monitor and every approval gate in the building reads this change as an improvement. There is no distribution shift to detect, because nothing drifted. A column arrived.

The governance check has nothing to say

Asked under US employment law, the tag-based check returns two findings that were already there yesterday and says nothing about the new column. It is not broken and it is not wrong. Insurance coverage is genuinely not a protected characteristic, so a check that reads labels has nothing to report.

Two questions, asked separately

Ariadne asks two different kinds of question about a model, and keeps them apart on purpose. One is about labels and structure. The other is about information. Each catches things the other cannot.

Question one: is this model wired to something it should not be

A structural invariant is a provable fact about the graph, not a threshold on a metric. It asks whether a column tagged as restricted flows into a feature of a model the registry says is serving. Both halves matter: a protected column sitting in a warehouse table is normal, and the same column reaching a live decision is what regulation cares about.

Deployment is read from the registry, never inferred. The **PROD** in a model identifier is the environment the entity lives in, not a statement that anything is serving, and treating it as one would report every archived experiment as live.

Question two: what can the model rebuild that it does not contain

Removing a protected column is the control most teams believe they have applied. It is only a real control if the remaining columns do not carry the same information. Income, education, occupation and neighbourhood each correlate with race, and enough of them together reconstruct it.

So Ariadne measures it directly. Drop the attribute, fit an ordinary model to predict it from everything that remains, and score on held out rows. The output is a number that can be shown to somebody who has to sign off, and checked by somebody who disagrees.

Watch the change, not the level. Race sits at 0.73 from an ordinary lending model. Is that bad? Compared to what? Education correlates with race because society does, and no threshold on the level separates that from a fault in the pipeline without an argument. But if the number sits at 0.73 for a year and reaches 0.82 the week somebody edited a dbt model, that is not a fact about society. Something entered the pipeline.

The threshold is measured, not chosen

Refitting on a different split moves the number by a standard deviation of about 0.003 on this data. The threshold is four times the observed spread with a floor of 0.02, so roughly seven times the noise. The disability move was 0.0394, twelve times the noise. Nothing was tuned to make the demonstration fire.

Following it back

The alarm says which attribute moved and which columns the model gained. The graph says where the column came from. Every edge below was produced by a DataHub ingestion connector reading Postgres, the dbt manifest and the MLflow registry. Nothing was hand written, which matters because a graph you wrote yourself proves nothing about whether the approach works.

```
raw_person.pubcov    census extract
  ↓ postgres connector
stg_person.public_coverage_flag    renamed and cleaned
  ↓ dbt connector, column level edge
dim_person.public_coverage_flag    [personal_data]
  ↓ dbt connector, column level edge
workforce_features.public_coverage_flag    no tag here
  ↓ mlflow connector, table level edge
workforce-classifier_3    mlflow_production
```

Two things fall out of that chain. The governance tag exists one hop upstream of the feature table and not on the feature table itself, which is the entire gap. And the column is called something else where it enters, so nobody searching the warehouse for `public_coverage_flag` would ever have found `pubcov`.

The same column, found twice by different means

Which columns count as restricted is declared in a policy pack rather than written into the checks, so the same graph and the same tags produce different verdicts depending only on which statute is named.

REGIME	WATCHES	VERDICT ON THE COVERAGE FLAG
employment_us	columns tagged as protected attributes	nothing to report
ecoa	protected bases under fair lending	nothing to report
eu_ai_act	protected attributes and personal data	examine, Article 10(2)(f)

So the column is caught twice. Once by measuring information, needing no labels at all. Once by reading a tag, needing no statistics. Two independent routes reaching the same column is the closest thing to corroboration this problem allows.

Before the change, and after it

Two agents, one graph, opposite directions. Neither has a language model in its decision path. That is a choice rather than an omission: the answers are traversals and measurements, so they reproduce exactly, and somebody who disagrees can check them instead of arguing with them.

Blast radius, at the pull request

Root cause is the glamorous half and the wrong half to reach for first. By the time anyone runs it, the model is already serving decisions off the thing nobody meant to add. The cheap moment is the pull request, when the change is still a diff and the only cost of knowing is a comment.

```
$ python tools/blast.py analytics_marts.dim_person --column public_coverage_flag --policy eu_ai_act
== blast radius of a change to analytics_marts.dim_person.public_coverage_flag ==
    read through kit, under Regulation (EU) 2024/1689, Annex III high risk systems

    examine      in scope because it carries personal_data, and EU AI Act watches that tag.

reaches 1 deployed model
    workforce-classifier_3      mlflow_production, 5 hops

and 1 registered but not serving
    workforce-classifier_1      mlflow_archived
```

The split between deployed and merely registered is what makes it readable. A change reaching forty registered models and no live one is a completely different change from one reaching a single live model, and a plain downstream count cannot tell them apart.

Root cause, after the fact

It starts from the measurement rather than from the diff, because plenty of columns get added that change nothing and listing them all would be a changelog rather than a cause.

```
$ python tools/rootcause.py --model workforce-classifier --policy eu_ai_act --via mcp
accuracy went up, 0.6955 to 0.7310 (+0.0355)
so nothing that watches model quality had a reason to object

1 measurement moved beyond noise
    disability (With a disability against Without a disability)
    0.7489 to 0.7883 +0.0394, 12x the noise

    public_coverage_flag
        enters at postgres:raw_person, 7 hops back
        called pubcov where it enters, so a search by name would not have found it
        defined in models/marts/workforce_features.sql

cause: public_coverage_flag
```

A column, a file, a moment and a rename in one answer. Where it cannot tell two causes apart, it names both and says so. A confident single answer would read better and would be invented.

Built on the platform, not beside it

Every claim Ariadne makes is a claim about the DataHub graph, read through DataHub's own agent surfaces and written back through DataHub's own APIs.

SURFACE	WHAT IT DOES HERE	RESULT
Context graph	Column level lineage, back to the raw extract and forward to whatever is serving	7 hops
Ingestion connectors	postgres, dbt and mlflow. Every edge came from one of them	3 connectors
MCP Server	The read path an outside agent would take, as a child process over stdio, tools discovered at the handshake	6 tools
Agent Context Kit	The in process read path, and the runtime both agents are built on	both agents
Analytics Agent	Its context platform pattern rather than its application: one abstraction per transport, built by a factory	pattern adopted
DataHub Skills	An ML impact skill covering the question the existing five do not	1 authored
Incidents API	Findings written back where a data team already looks	9 raised
Tags and metadata	Governance tags declared in dbt, read back through the sibling graph	14 columns

Two transports, checked against each other

The Agent Context Kit runs in process and is fast. The MCP server is a child process spoken to in JSON-RPC, slower, and the honest one, because it is the path a third party agent would actually take. Two read paths are only worth having if they agree, so the same walks run down both and the results are compared: **three of three identical**. A fast path that quietly returns a shorter answer is worse than no fast path, because every check built on it reports all clear on a smaller graph.

The finding lands where somebody will see it

A governance tool that prints to a terminal is a governance tool that gets run once. Nine incidents are live in DataHub, all active, filed on the two feature tables. They appear next to the table in the catalog and show up as a health signal on any lineage walk that passes through, so a later investigation into something else finds the earlier finding on the way past.

Declare the hypothesis before measuring it

A plausible story about a column is not evidence about a column. This section exists because the first attempt was a guess and the guess was wrong.

Vehicles per household went into a feature table because it sounded like a disability proxy: two of the six census disability questions concern mobility, so vehicle access seemed a reasonable stand in. It moved disability by 0.0096, well under the threshold. It moved race, marital status and citizenship instead, and accuracy went up, so every metric said ship it.

So candidates are now written down first, each with the source of the claim and why it should hold, and only then measured. Recording it beforehand is what stops the screen from becoming a search for whichever column happened to score well, which would find something every time and mean nothing.

CANDIDATE	BASIS OF THE CLAIM	ALONE	ADDS	
PUBCOV	derived, the union of four public programmes	0.7020	+0.0544	fires
HINS4	definitional, the survey question names disability	0.6650	+0.0379	fires
SSIP	eligibility rule, SSI is paid on disability or age	0.5923	+0.0261	silent
HINS3	eligibility rule, working age Medicare is disability driven	0.5948	+0.0239	silent
VEH	guess, no source, kept as a control	0.6201	+0.0096	silent
POVPIP, GRPIP	correlational, through the income channel	—	—	not in this warehouse

244,328 rows, three seeds. The baseline rebuilds disability at 0.7321 before any candidate is added, and the threshold is 0.0222, the same one the alarm fires on.

The strongest evidence had no statistics in it. The candidate that fires hardest after the union is Medicaid, and the reason is the wording of the question that collects it: *"Medicaid, Medical Assistance, or any government-assistance plan for those with low incomes or a disability"*. A respondent living with a disability is being asked, in the same sentence, whether they are the kind of person the programme is for. The association is built into the instrument, and that is far more durable than an effect size from one paper.

Medicare is the useful failure. It had the better source of the two eligibility rules and it is silent anyway, because the features already present carry most of what it knows. It stays in the register as refuted rather than being deleted, as does the vehicles guess. A register holding only the hypotheses that worked would be a results table.

What if nobody tagged the warehouse

Everything above leans on somebody having tagged the columns, which is the weakest assumption in the design. The columns nobody thought about are exactly the ones that cause trouble. So the question runs the other way too: given a table and no tags at all, which columns hold a human attribute?

Three rungs, cheapest first, and the escalation is the point. The name is a free string comparison and wrong the moment somebody renames a column. The value distribution is one query and works on a column called `q4_resp` as well as one called `sex`. Fitting a model is expensive and is the only rung that finds a column carrying an attribute it does not contain.


```
$ python tools/identify.py public.raw_person
no tags read, no lineage walked. names and values only.

identified  rac1p  race      identified  agep  age
identified  dis    disability identified  pobp  place_of_birth
identified  sex    sex      identified  anc1p  ancestry
identified  cit    citizenship identified  mar    marital_status
identified  nativity nativity

ambiguous  pubcov  disability, nativity, sex
          several attributes share this code set and the name does not
          choose between them

9 identified, 3 ambiguous, 12 with nothing to say
```

Nine of nine, from names and value distributions alone, on codebook abbreviations with no tag ever written. And the coverage flag is honestly ambiguous at the cheap rungs, because it is coded one and two exactly like sex and disability, so it is handed up to the expensive rung rather than guessed at. That expensive rung is the one that found it in the first place.

09 · LIMITS

Reading this honestly

Four things this does not do, stated here rather than left for somebody to find.

1. **It does not decide legality.** Correlation with a protected attribute is not prohibited and cannot be, because income and education correlate with race, and a model stripped of everything correlated with race predicts nothing at all. What the law asks is whether a practice causes harm, whether the business need justifies it, and whether a less harmful alternative exists. Those are human decisions. This supplies one input.
2. **It is not fully label free.** Knowing that [disability_code](#) means disability came from a declared policy file. Lineage carries that one fact everywhere afterwards, so you have to be right once, but once is not zero.
3. **Public coverage is not purely disability.** In California after Medicaid expansion it also tracks low income. Medicare below sixty five is the cleaner channel and is weaker here. The register records both.
4. **The level is arguable, the change is not.** This is why the alarm watches movement against measured noise rather than asserting a threshold on the level, and why every finding names the columns that arrived alongside it.

The dataset is de-identified, and that is fine

ACS PUMS carries no names or emails. Personal data here means protected and personal attributes tagged under a governance policy, which is what real governance teams manage and what fair lending law regulates. The failure mode is native rather than contrived: the canonical [folktables](#) income task ships with race and sex in its standard feature list, so no scenario had to be invented.

Four things that were not in the documentation

Each one fails silently, which is why they were worth writing up rather than working around quietly. A loud failure teaches you something the first time. These teach you nothing until much later.

- 01 The MCP server stalls before the handshake.** It posts a usage ping before answering `initialize`. Where that host is unreachable the send retries for about forty seconds, and because it happens before the protocol starts, the client sees no error and no response at all. Common on air-gapped hosts and on any cloud instance whose egress rules allow the catalog endpoint but not general outbound traffic. [datahub#18684](#)
- 02 Incidents do not attach to an ML model.** The documentation said "dataset, dashboard, chart, dataFlow, etc" and the "etc" is not open ended. Seven entity types carry the aspect and no ML entity is among them. The feature table turned out to be the better home anyway, since that is where a fix goes. [datahub#18685](#)
- 03 An ML impact skill.** The registry covers lineage, search, enrichment and quality. None cover whether a warehouse change reaches something that is serving. The skill carries all four traps found here, so an agent following it gets the answer rather than the plausible version of it. [datahub-skills#66](#)
- 04 The Agent Context Kit cannot import its own registration module** against the dependency version it pins, so the LangChain and Google ADK paths fail on a clean install. The tool functions are unaffected and were used throughout. [datahub#18686](#)

And two the build had to survive

Ingest order changes what the graph contains. The mlflow connector overwrites the warehouse entity's schema with the training frame's columns, so postgres has to run after mlflow. The dbt source resolves `select *` against the schema DataHub already holds, so postgres also has to run before dbt. The order is postgres, dbt, mlflow, postgres. One column tracing and nine not is a bug you find. Nine tracing and one not is a bug you ship.

A table exists three times and only one copy has the tags. dbt, the warehouse and the ML platform each hold their own entity, joined as siblings. Governance tags land on the dbt entity and lineage traversal returns the warehouse one. Ask the wrong sibling whether a column is protected and the answer is no, with no error. This is the most likely way to ship a governance check that looks like it works and does not.

How it is put together

LAYER	WHAT IT IS
pipeline/extract.py	Pulls census person, household and geography records into Postgres
warehouse/	dbt project. Staging renames codes, marts join and derive, feature tables feed models
ml/train.py	Trains and registers the classifiers, logging the feature list and the source table
ingest/	DataHub recipes for the three connectors
policy/	Declared regimes. Attributes, bases, citations. Three statutes over one warehouse
tools/context.py	The read layer. Two transports behind one interface, tools discovered not assumed
tools/trace.py	Column traversal, sibling resolution, restricted attribute reachability
tools/sentinel.py	Structural invariants over the graph
tools/reconstruct.py	Measures what the features can rebuild that the model excludes
tools/exposure.py	Records that measurement over time and fires when it moves past the noise
tools/blast.py	Forward walk before a change lands. Emits a pull request comment
tools/rootcause.py	Backward walk after one did. Names the column, the file and the moment
tools/screen.py	Measures the declared proxy register
tools/identify.py	Works out what a column holds with no tags at all
tools/incident.py	Writes findings back into DataHub. Nothing without an explicit flag
tools/agree.py	Checks the two read paths return the same graph
research/proxies.yml	The hypothesis register, with sources, refuted entries kept
state/exposure.json	Measurement history, which doubles as the record keeping trail

Running on a single EC2 instance: DataHub v1.5, Postgres, MLflow and dbt. The census data is pulled live from the Census API and nothing is redistributed, which keeps the licensing position clean. Apache 2.0 throughout.

Where to look first. The recorded run, start to finish, with every command and its unedited output, is at [ariadne-five.vercel.app](#). Each step links to a page proving the DataHub surface it used. The code is at [github.com/cnpierrepapi/ariadne](#).